



Technical Assignment

Convolutional networks and vision-language models for embodied intelligence

ROLE	AI / VLM Engineer — AI & Foundation Models
DEADLINE	5 calendar days from receipt of this document
COMPUTE	Scoped to a single consumer GPU (Google Colab free / T4, or an RTX 3090–4090 class card). No cluster required.
SUBMIT TO	<code>careers@etrobotics.ai</code>

Congratulations on being shortlisted. This assignment is designed to show us how you think, not how much you can write. We are far more interested in reproducible code, honest plots and clear reasoning about your own results than in polish. Please do not gold-plate — if something did not work, say so and explain why.

PROBLEM 1

MNIST — CNN with an MLP head

Download the open source MNIST data set and implement the following, you need to complete the below mentioned tasks:

1. Forward propagation and back propagation for a simple CNN with a MLP head, with justified choice of layers, number of neurons in each layer and activation functions.
2. Justification for the achieved F1 score, precision, recall and accuracy with proper plots.
3. Verify the stochastic gradient approximation convergence conditions — ergodicity of input data, step size conditions, unbiased gradient, bounded variance and L smoothness. If not verifiable then give proper reason for that.

PROBLEM 2

Fine-tuning nanoVLM on Robo2VLM-1

Download the open source Robo2VLM-1 **dataset** (Apache 2.0). In each training example we have five columns — id (string, encoding the source trajectory and question number), question (string), choices (a stringified Python list, not a list), correct_answer (integer index into choices, 0–4), and image (single JPEG, 84 to 640 px wide). Roughly fifteen question templates covering depth ordering, next-move direction, task success, cross-camera point correspondence, goal-state selection, and instruction matching. Ground truth is derived from robot sensor channels, not human annotation.

Fine-tune the pre-trained open source nanoVLM vision-language model on it, you need to complete the following tasks:

1. Convert the multiple-choice annotations into a vision-language training format suitable for nanoVLM.

2. For model evaluation refer to the points below (plot graphs wherever required):
- a. Report at least 5 evaluation metrics for the trained VLM, the metrics should be standard evaluation metrics used in VLM / LLM evaluation (like perplexity, Cohen's Kappa, etc). Properly justify the choice and the results of the evaluation metrics.
 - b. Implement a proper attention map (or saliency map), to understand the important parts in input image.
 - c. Implement top-k, temperature scaling, top-p nucleus sampling and min-p sampling. Create a table explaining the effect of these on your output.
 - d. Establish the baseline before fine tuning, compare your results with this baseline checkpoint.

SUBMISSION

What to send us

Please send a single email to careers@etrobotics.ai within 5 calendar days of receiving this assignment, with the subject line:

AI/VLM Engineer – Assignment – <Your Full Name>

INCLUDE

- A link to a **public or access-shared Git repository** containing all code for both problems. Two clearly separated folders (e.g. problem1_mnist/ and problem2_nanovlm/) with a README explaining how to reproduce your results end to end.
- Notebooks or scripts we can actually run, plus the exact environment (requirements.txt / environment.yml) and any seeds needed for determinism.
- **Raw training logs** — Weights & Biases links or TensorBoard event files, not only screenshots — along with the plots you reference in your write-up.
- A short write-up (PDF or Markdown, **4 pages maximum**) that answers every numbered task above and cites your own numbers. State your hardware, wall-clock training time and anything you would do differently with more compute.
- If a task could not be completed, keep it in the write-up and explain what you tried and where it broke. This scores far better than silence.

HOW WE WILL READ IT

We will clone your repository and attempt to reproduce at least one result from each problem. Generic write-ups that could have been produced without running anything will not score. Honest negative results, clear justifications and readable code carry the most weight.

Use of AI coding assistants is fine and expected — but you must be able to explain and defend every line of what you submit in a follow-up discussion.

QUESTIONS

If anything is ambiguous, make a reasonable assumption, document it in your README, and carry on. If you are genuinely blocked, write to **careers@etrobotics.ai** — asking a good question is never held against you.

We are looking forward to seeing what you build.